Binary Search Algorithm Guide

Comprehensive overview, step-by-step example, and Python implementation

Introduction & Overview

Binary search is an efficient algorithm used to find the position of a target value within a sorted array. Instead of checking every element sequentially (like a linear search), binary search repeatedly divides the search space in half, dramatically reducing the number of comparisons needed.

- **Prerequisite:** The input array or list **must be sorted**.
- **Time Complexity:** $O(\log n)$ — extremely scalable for massive datasets.
- **Space Complexity:** $O(1)$ for the iterative implementation.

Step-by-Step Example

Let us trace the algorithm with a concrete example. Suppose we have a sorted array of integers and we want to find the target value $target = 23$.

Index	0	1	2	3	4	5	6
Value	2	5	8	12	16	23	38

Iteration 1

- **Low Index:** 0 (Value: 2)
- **High Index:** 6 (Value: 38)
- **Mid Index:** $(0 + 6) / 2 = 3$ (Value: 12)
- **Comparison:** Our target (23) is greater than the middle element (12). We discard the left half and update $low = mid + 1$ (Index 4).

Iteration 2

- **Low Index:** 4 (Value: 16)
- **High Index:** 6 (Value: 38)
- **Mid Index:** $(4 + 6) / 2 = 5$ (Value: 23)
- **Comparison:** The middle element at index 5 matches our target (23). **Target found at index 5!**

Python Implementation

```
def binary_search(arr, target):
    low = 0
```

```
    high = len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        # Check if target is present at mid
        if arr[mid] == target:
            return mid

        # If target is greater, ignore left half
        elif arr[mid] < target:
            low = mid + 1

        # If target is smaller, ignore right half
        else:
            high = mid - 1

    return -1

# Example usage:
arr = [2, 5, 8, 12, 16, 23, 38]
target = 23
result = binary_search(arr, target)
print(f"Element found at index: {result}")
```